



MALWARE

Understanding and Mitigating Threats

Information Systems — BSc Program

Academic College of Tel Aviv-Yaffo

Topic 2

Malware



- Software are instructions that tell a computer how to work

Malware is a category of software which is **hostile** and/or **intrusive**

- Malware = Malicious + Software



it could perform hostile activities – deleting data or files, attacking other computers, or performing intrusive actions you do not consent to.

What is Malware?

Malicious Software (Malware) — any program or code designed to infiltrate, damage, or perform unauthorized actions on a computer system. It can delete data, steal information, hijack resources, or attack other computers without the user's consent.



Data Theft

Exploits vulnerabilities to steal passwords, financial data, and personal information — leading to identity theft and financial loss.



Service Disruption

Overwhelms network services (e.g., DDoS), causing downtime, business interruptions, and loss of customer trust.



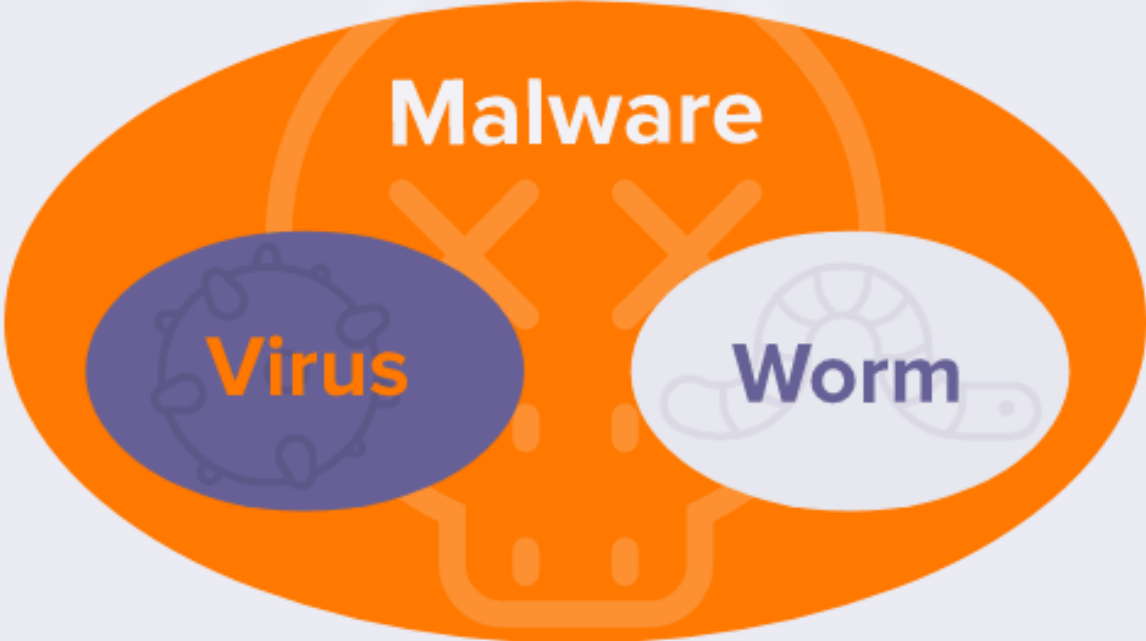
System Damage

Corrupts files, degrades performance, causes crashes, and may render systems completely inoperable.



Espionage

Silently monitors user activity, captures keystrokes, and exfiltrates sensitive corporate or government data.



A diagram illustrating the relationship between Malware, Virus, and Worm. A large orange oval labeled "Malware" contains two smaller ovals: a dark blue one labeled "Virus" and a light blue one labeled "Worm". The "Virus" oval features a spiky virus particle icon, and the "Worm" oval features a segmented worm icon. The background of the entire diagram is light blue.

Malware

Virus

Worm

Viruses

A program that attaches to a host file and requires human interaction to execute and spread.

Document-Based Virus (e.g., .DOC)

1. User downloads an infected document
2. Document prompts user to enable macros
3. Enabling macros triggers the embedded virus code
4. Virus executes its payload — data theft, corruption, or propagation

Executable-Based Virus (e.g., .EXE)

1. User downloads and runs an infected executable
2. Virus activates immediately upon execution
3. Commandeers system resources to replicate
4. Spreads to other files and connected devices

Key Insight: A virus always needs a host file and user action to activate.

Virus

- ...Inserts **itself into other computer programs**

Microsoft Word is the **host** program for the virus



- When the host program is run, the virus code executes too
- When this happens, the virus will replicate itself and spread

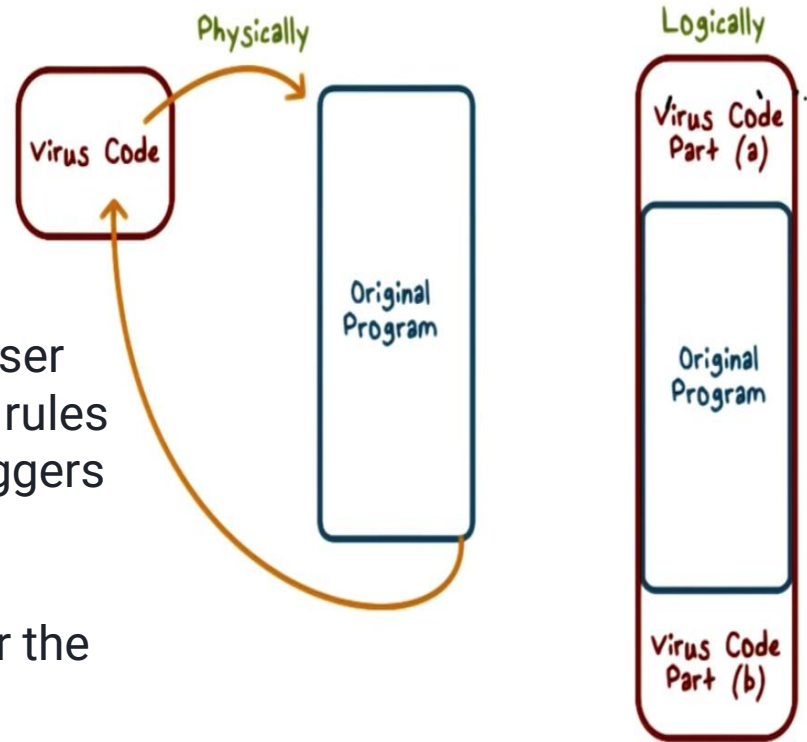


Viruses

A computer virus lives within a host, such as a document or executable file, and requires human interaction to spread.

If it's a DOC file, once the user downloads it, the user will be prompted to enable macros (programmed rules that help simplify repetitive tasks). This action triggers the virus.

If the file is an EXE, downloading it and running, it triggers the virus. The virus will then commandeer the computer's resources to copy itself and spread, damaging your devices and files or stealing data.



Virus Propagation Methods

01



File Sharing

Infected files spread through email attachments and removable drives (USB).

02



Malicious Downloads

Users download infected files from untrusted websites or software repositories.

03



Web Exploitation

Visiting compromised websites triggers automatic execution of malicious code.

04



Social Engineering

Users are tricked into opening infected files disguised as legitimate content.

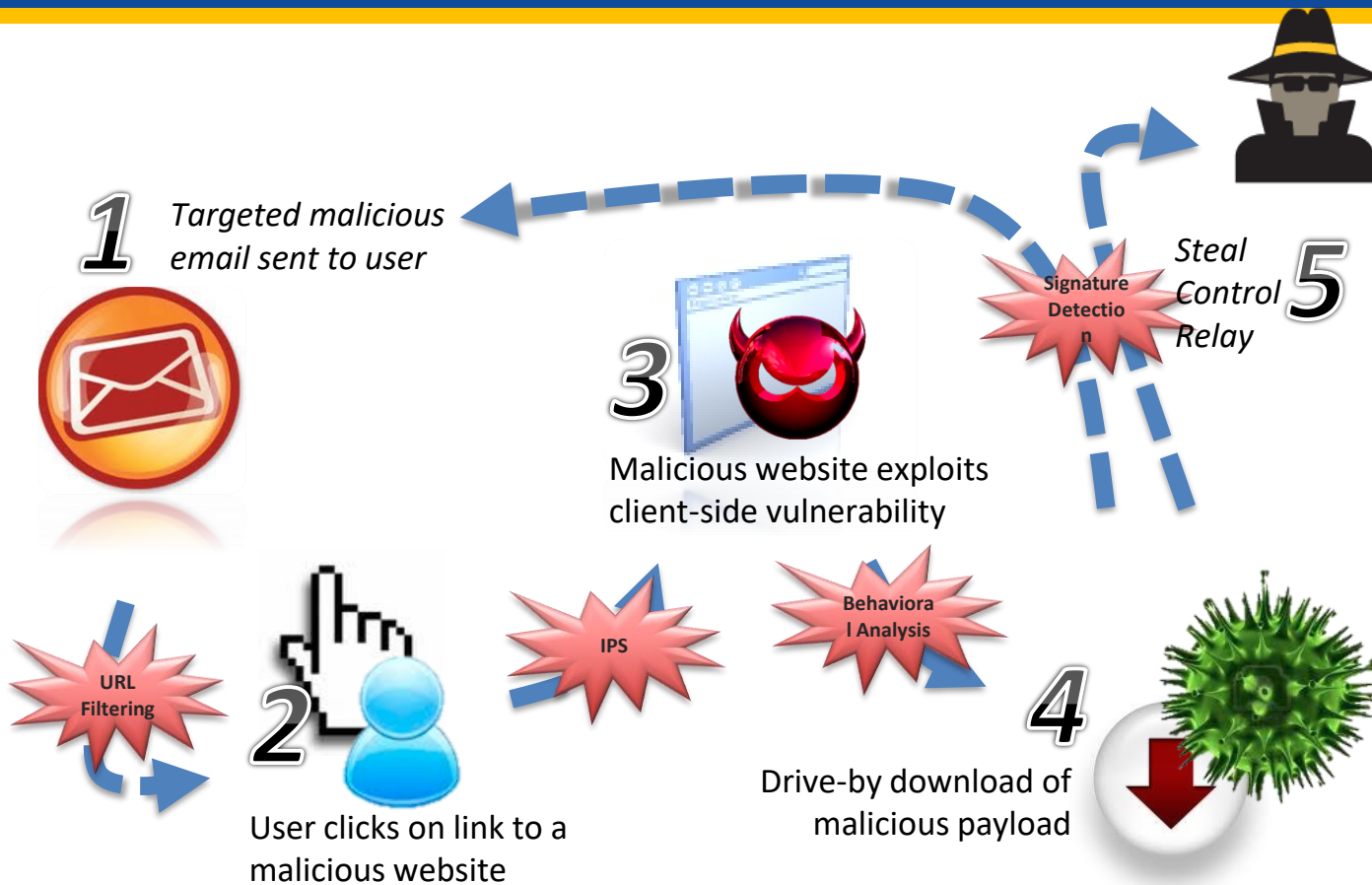
05



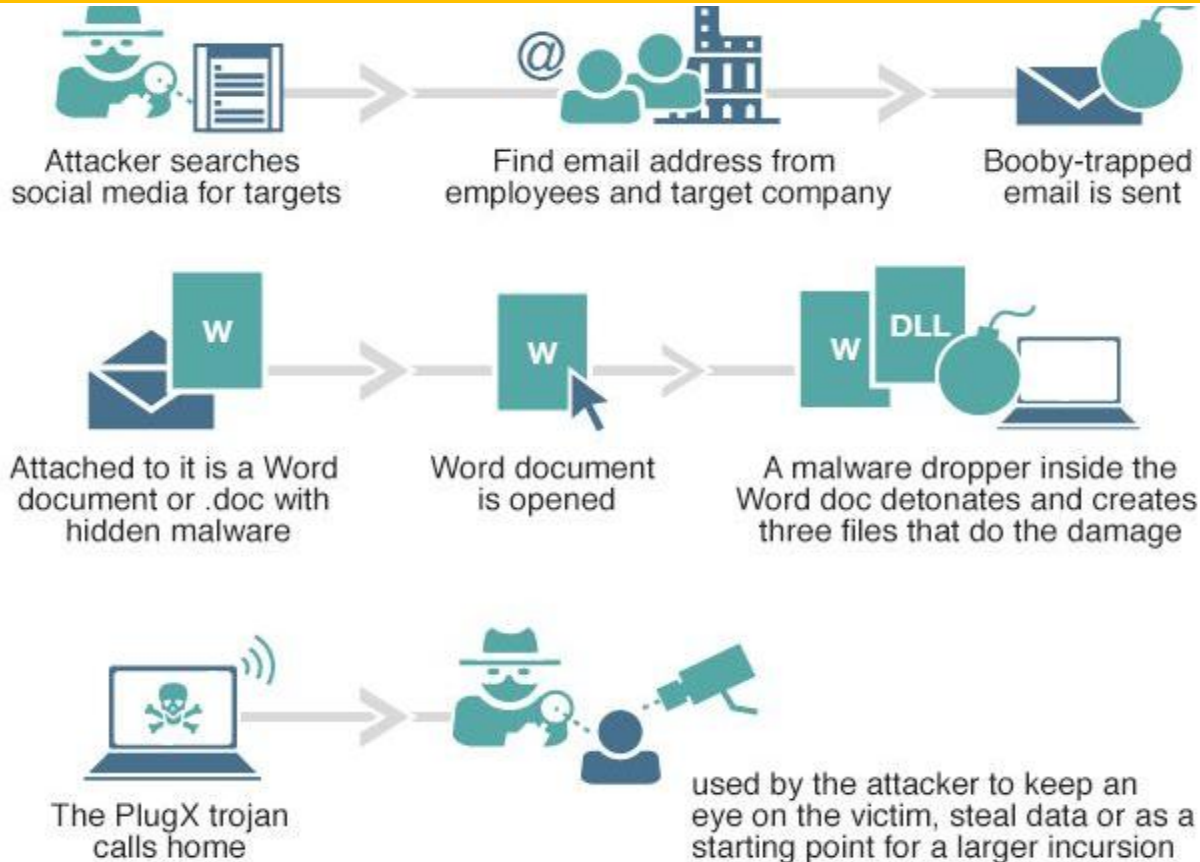
User Activation Required

Viruses always require some form of user interaction to execute.

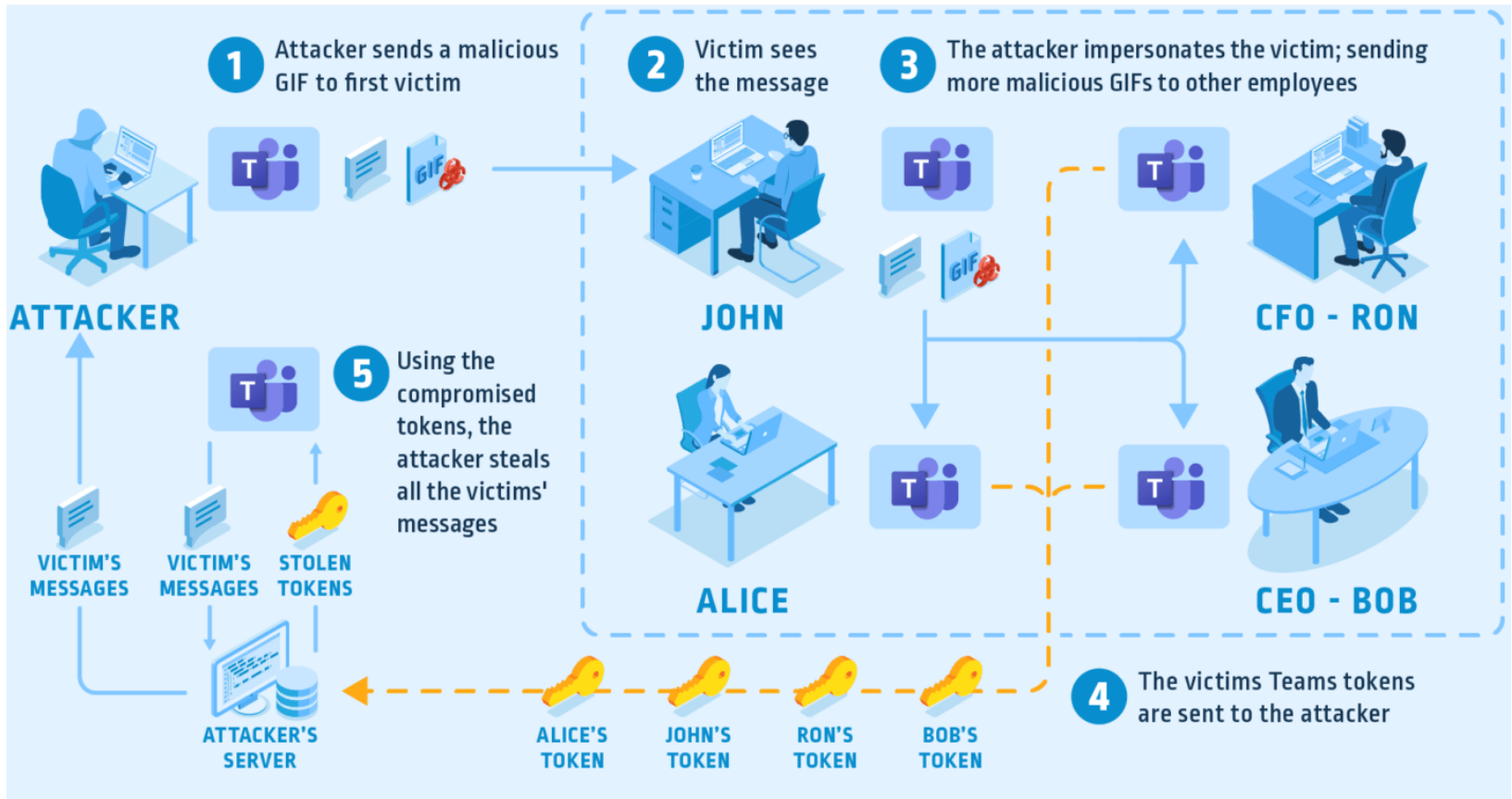
Attack Example



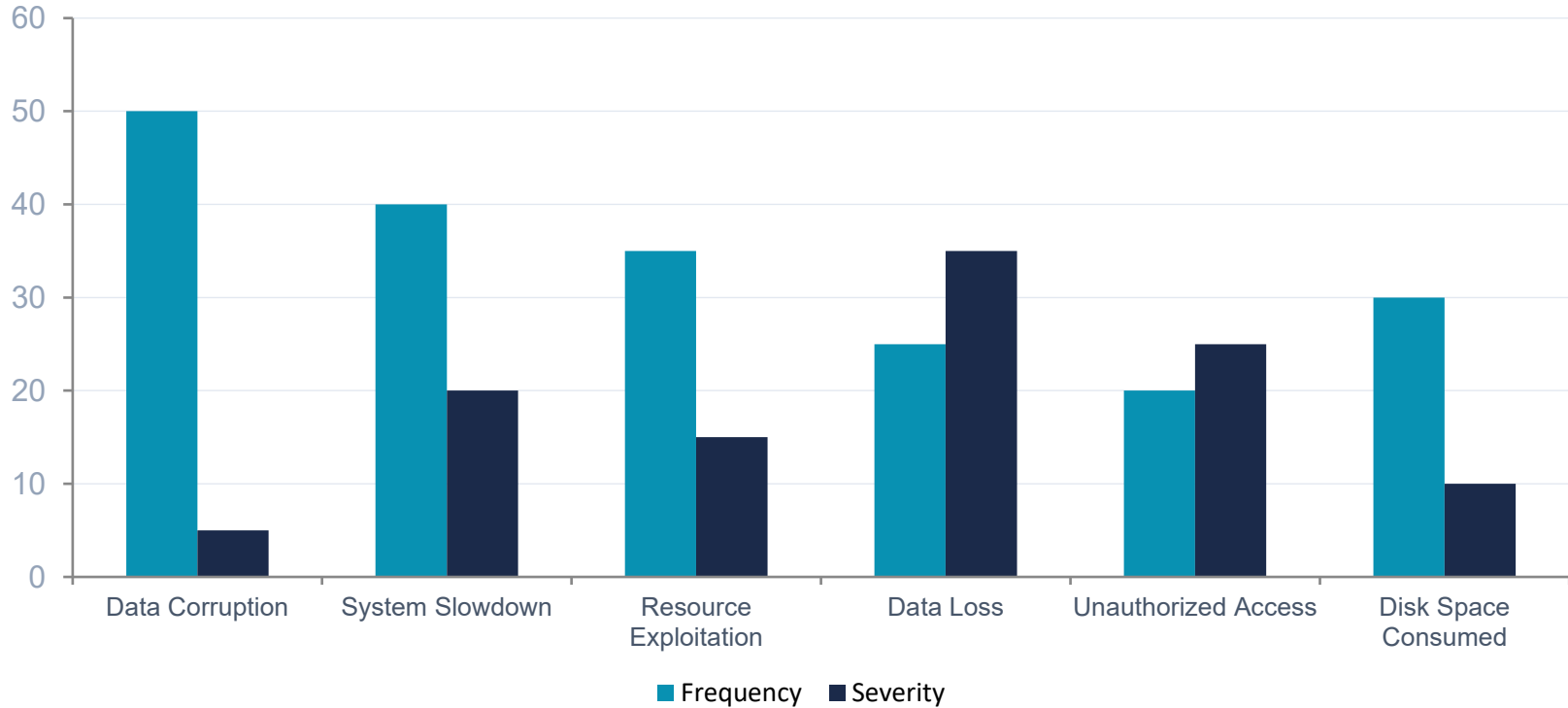
The anatomy of a virus attack



The anatomy of a virus attack

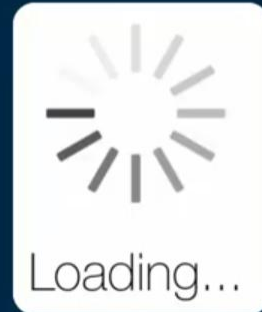


Impact of Viruses on Systems



Worms

- *Unlike* a virus, a worm does not need a host program
- They exist as a standalone (e.g. .exe) program
- *Like* viruses, worms can replicate very quickly
- Many worms work over **networks**, and spread between computers and slow down communication



Worms

Autonomous malware that self-replicates across networks without user interaction.



Self-Propagation

Spreads across networks by exploiting OS and application vulnerabilities — no user action required.



Resource Exploitation

Consumes CPU, memory, and bandwidth, causing severe slowdowns and potential system crashes.



Data Exfiltration

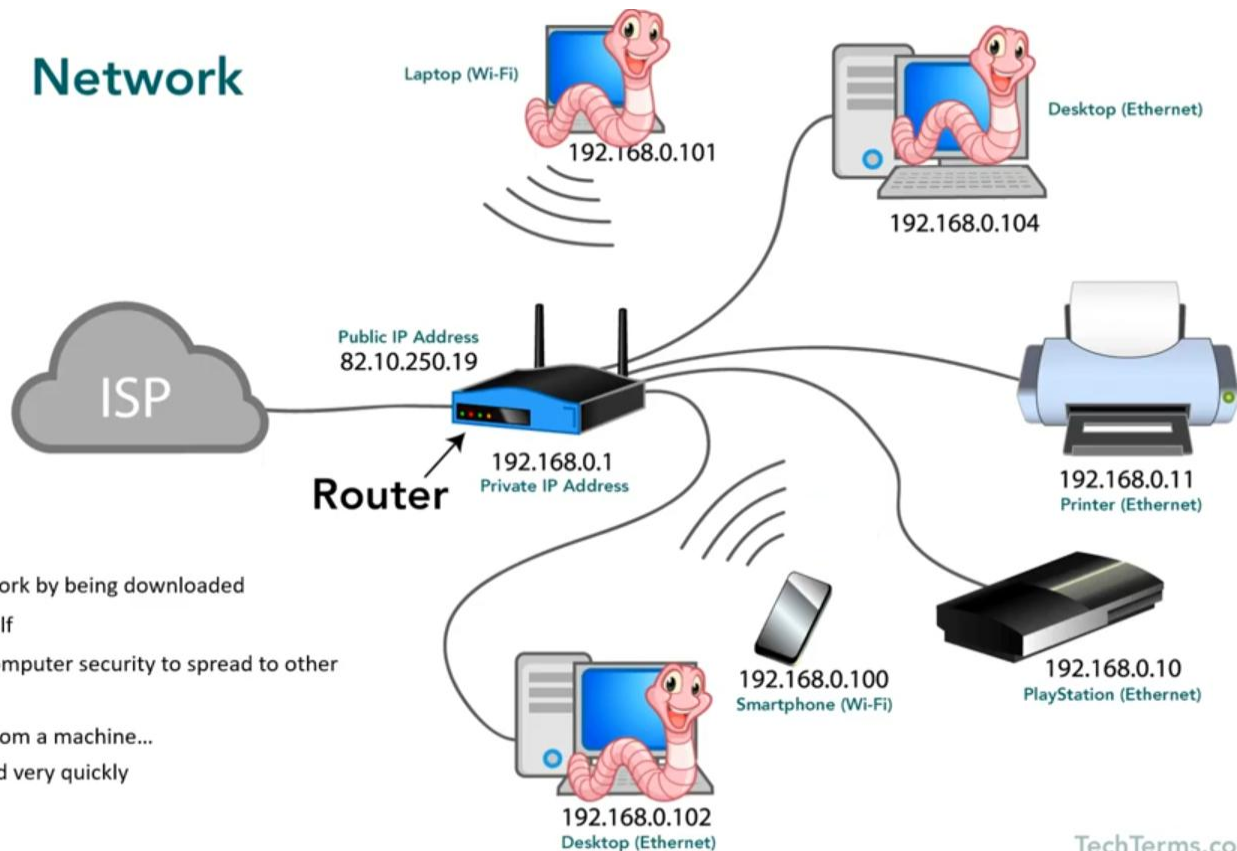
Designed to silently access and steal sensitive data from compromised systems.



Network Disruption

Overwhelms bandwidth and disrupts legitimate communication and service availability.

The anatomy of a worm attack



A worm is a program

It can get onto the network by being downloaded

Once there it copies itself

It uses weaknesses in computer security to spread to other machines

If you remove a worm from a machine...

... it often gets reinfected very quickly

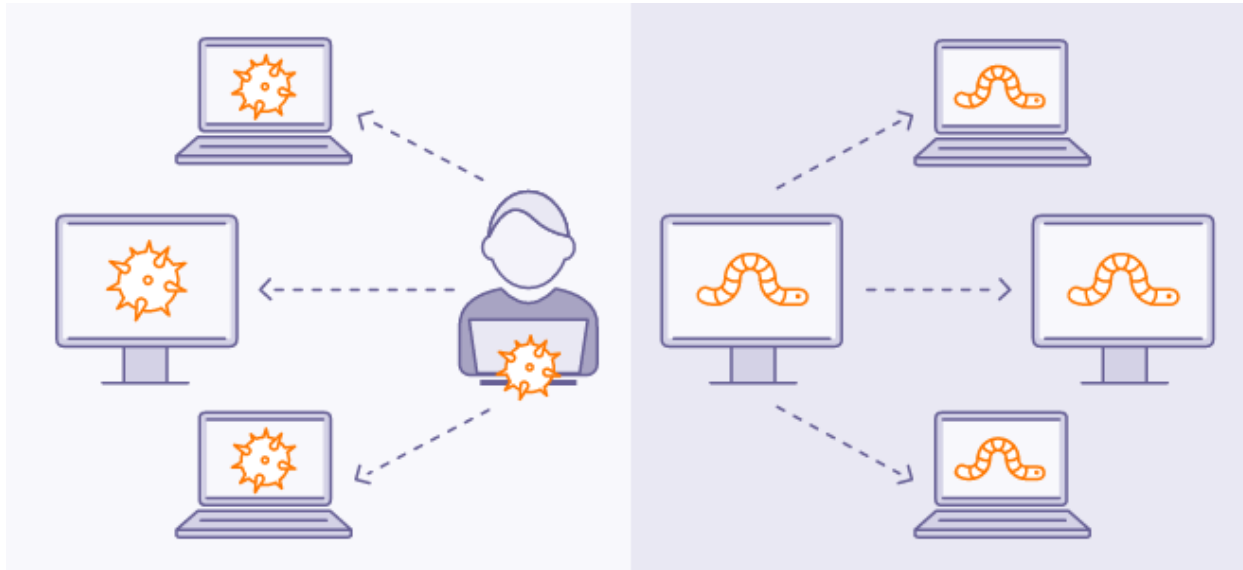
Worms vs. Viruses

Attribute	Worm	Virus
Propagation	Spreads independently across networks	Requires user action to spread
Host Dependency	No host file needed	Needs a host file or program
Activation	Triggers automatically via vulnerability exploit	Requires user interaction with infected file
Speed	Rapid replication — can infect thousands in minutes	Slower spread — dependent on user behavior
Example	WannaCry, Conficker, SQL Slammer	ILOVEYOU, Melissa, CIH

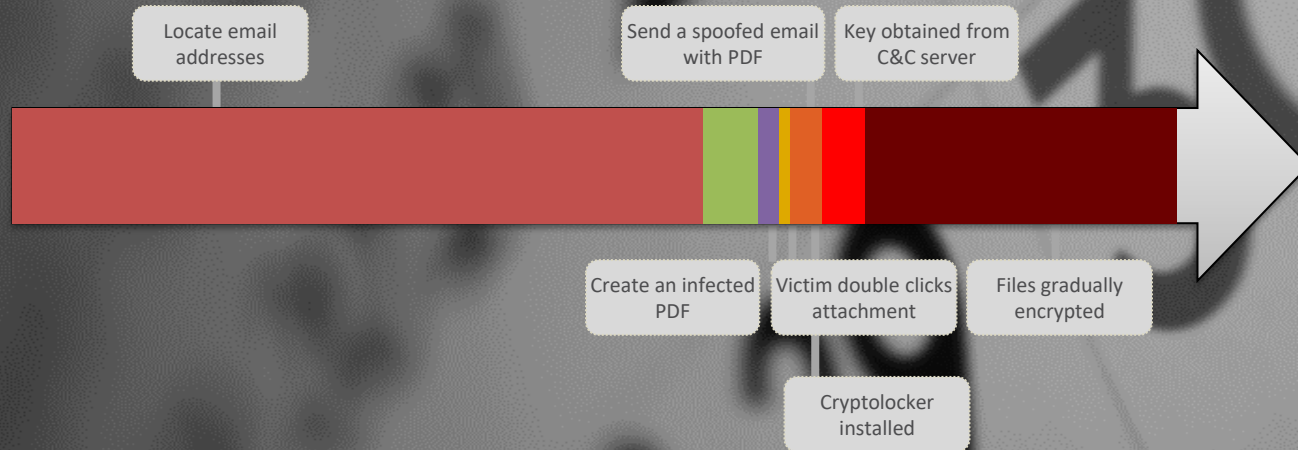
Worms are autonomous. They replicate and spread without any host activation or user intervention.

Worms vs Viruses

Unlike viruses, worms can replicate and spread without any host activation.

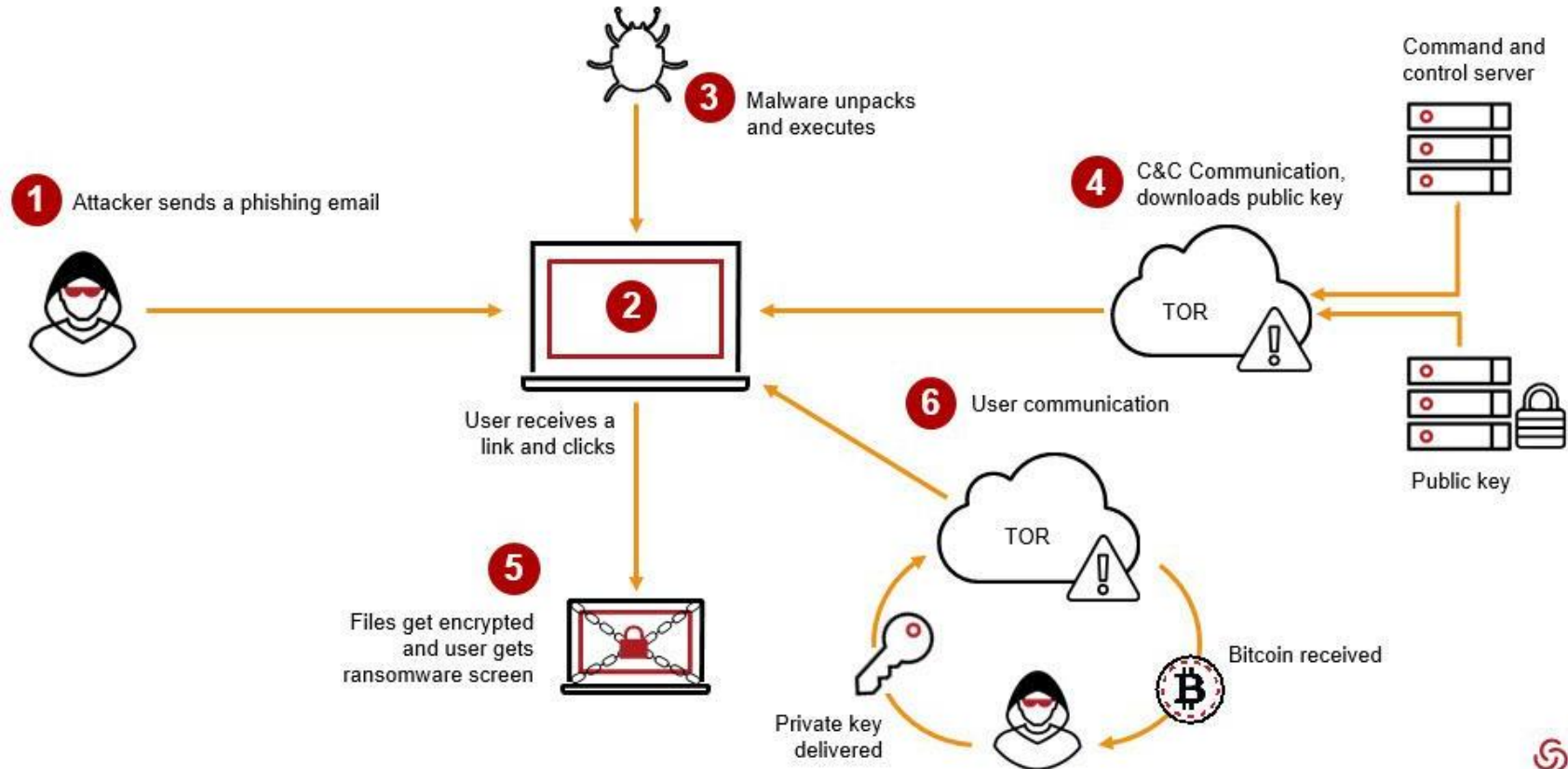


Ransomware



Some steps take hours or even weeks,
while others take mere seconds

The anatomy of a ransomware attack





Understanding Droppers

Malware Delivery Mechanisms

Information Systems – Cybersecurity Fundamentals

What Is a Dropper?

Definition

A dropper is a type of malware designed to deliver and install additional malicious software (the payload) onto a victim's system. It acts as a carrier, its sole purpose is to get the real malware past defenses and onto the target machine.



Single-Stage Dropper

Contains the payload embedded within its own code. No network connection required to deliver the malware.



Two-Stage Dropper

Downloads the payload from a remote server after execution. Also called a 'downloader.' More flexible for attackers.

The Dropper Attack Chain



Spear Phishing

Attacker sends a targeted email with a malicious link or attachment



User Clicks Link

Victim is tricked into clicking the malicious URL in the email



Dropper Downloaded

The dropper malware is silently downloaded to the victim's machine



Payload Deployed

Dropper installs the real malware (e.g. RAT, keylogger, rootkit)



C2 Beacons

Malware phones home to the attacker's command-and-control server



Remote Control

Attacker issues commands: steal data, log keystrokes, pivot laterally

The Dropper Attack Chain

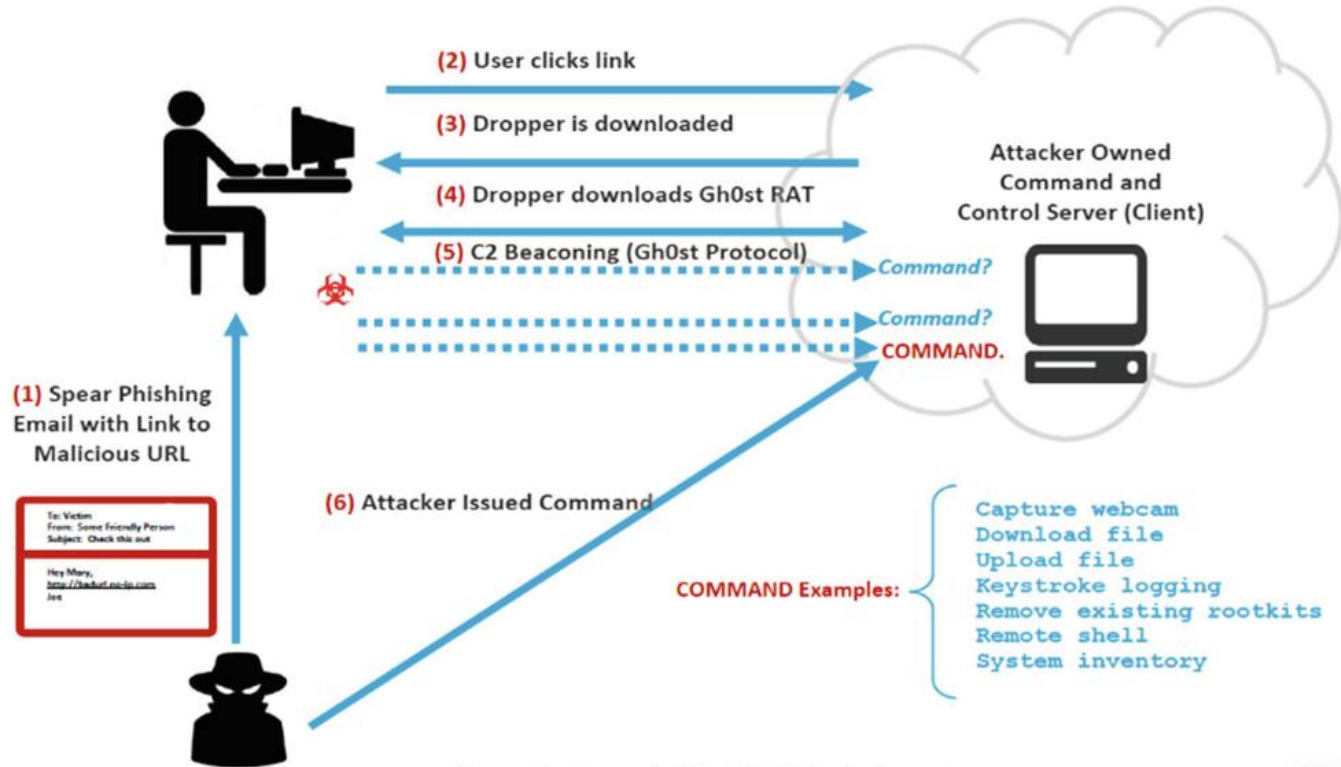


Figure 1 – Example Gh0st RAT Attack Scenario

How Droppers Evade Detection

Code Obfuscation

Encrypted or encoded payload makes static analysis difficult. The dropper decrypts the payload only at runtime.

Fileless Techniques

Some droppers inject payloads directly into memory, never writing to disk — bypassing file-based antivirus scans.

Legitimate-Looking Wrappers

Droppers disguise themselves as legitimate installers, PDFs, or Office documents with embedded macros.

Sandbox Detection

Advanced droppers detect virtual machines and sandboxes, delaying execution or behaving benignly when analyzed.

Polymorphism

Each copy of the dropper has a different binary signature, defeating hash-based and signature-based detection.

Staged Delivery

Downloading the payload in stages from rotating C2 servers makes it harder to trace and block the full chain.

Detection & Defense Strategies



Email Security Gateways

Filter phishing emails, scan attachments in sandboxes, and strip malicious links before they reach users.



Endpoint Detection & Response

EDR tools monitor process behavior, flag suspicious activity (e.g. a PDF spawning a cmd.exe), and enable rapid response.



Network Monitoring

Detect C2 beaconing patterns, unusual outbound traffic, and connections to known malicious infrastructure.



User Awareness Training

Train employees to recognize phishing attempts, suspicious attachments, and social engineering tactics.



Principle of Least Privilege

Limit user and process permissions to reduce the damage a dropper can do even if it executes successfully.



Threat Intelligence & IOCs

Use indicators of compromise (hashes, domains, IPs) to proactively detect and block known dropper campaigns.

Key Takeaways

1

Droppers are delivery vehicles — they exist to install the real malware.

2

The attack chain typically begins with social engineering (phishing).

3

Modern droppers use sophisticated evasion techniques (fileless, polymorphic, sandbox-aware).

4

Defense requires a layered approach: technology, training, and policy.

5

Understanding the dropper lifecycle is essential for incident response.



Web Shells

Understanding the Hidden Backdoor

Information Systems Security

What Is a Web Shell?



A web shell is a malicious script uploaded to a web server that gives an attacker remote access and control through a standard web interface (HTTP/HTTPS).



Script-Based

Written in server-side languages like PHP, ASP, JSP, or Python



Hidden in Plain Sight

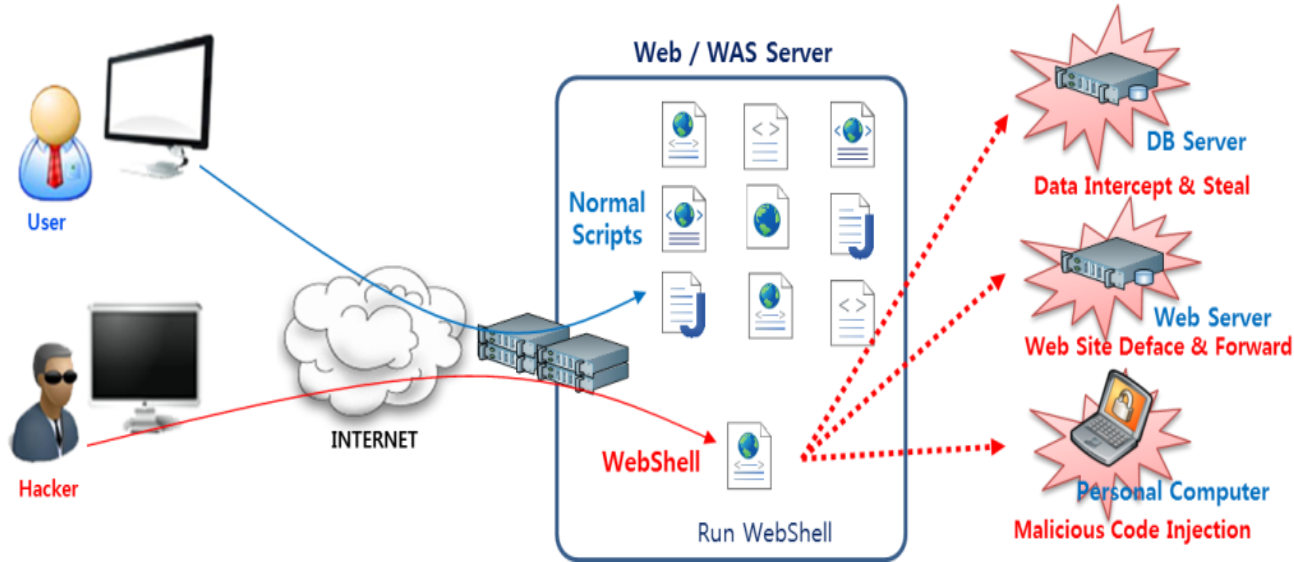
Disguised among legitimate files on the web server



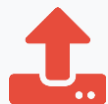
Remote Control

Provides command execution, file management, and data exfiltration

What Is a Web Shell?



How Do Web Shells Get Installed?



Unrestricted File Upload

Exploiting upload forms that don't validate file types, allowing .php or .asp files



Vulnerability Exploitation

Leveraging RFI, LFI, SQL injection, or known CMS/plugin vulnerabilities



Compromised Credentials

Using stolen FTP, SSH, or admin panel credentials to place shell files directly



Supply Chain Attack

Embedding shells in compromised third-party plugins, themes, or software updates

Web Shell Attack Flow



Potential Impact After Compromise



Data Theft

Access databases & sensitive files



Website Defacement

Modify site content & redirect users



Lateral Movement

Pivot to internal network systems

What Can a Web Shell Do?



Execute Commands

Run OS commands as the web server user



File Management

Upload, download, edit, and delete files on the server



Database Access

Query and dump database contents



Network Pivoting

Scan internal networks and attack other hosts



Credential Harvesting

Extract passwords from config files and memory



Persistence Mechanisms

Create backdoor accounts, scheduled tasks, and additional shells

Common Web Shell Examples

Name	Language	Notable Features
China Chopper	ASP, ASPX, PHP	Only ~4 KB; extremely hard to detect; widely used by APT groups
WSO (Web Shell by oRb)	PHP	Full-featured: file manager, DB client, brute-forcer, process viewer
b374k	PHP	Password-protected; file manager; command execution; self-delete
JspSpy	JSP	Database management; port scanning; reverse shell
ASPXSpy	ASPX	Targets IIS servers; file browser; command execution



Many web shells are open-source and freely available — attackers can customize them or write their own in minutes.

Detecting Web Shells



Indicators of Compromise

- ✗ Unexpected new files in web directories
- ✗ Files with recent modification timestamps
- ✗ Anomalous HTTP POST requests to static-looking files
- ✗ Unusual outbound network connections
- ✗ Web server process spawning cmd.exe or /bin/sh



Detection Methods

- ✓ File integrity monitoring (FIM) tools
- ✓ Web Application Firewalls (WAF) rules
- ✓ Endpoint Detection & Response (EDR)
- ✓ Log analysis for suspicious patterns
- ✓ Regular code review & static analysis

Prevention & Defense



Validate All Uploads

Whitelist allowed file types, scan content, store outside web root, and rename uploaded files



Harden the Server

Apply least privilege, disable unnecessary functions (eval, exec), keep software patched and up to date



Implement WAF Rules

Block known web shell signatures, suspicious parameters, and encoded payloads at the gateway



Monitor Continuously

Deploy FIM, analyze web server logs, set up alerts for anomalous file changes or process spawns



Segment the Network

Isolate the web server from internal systems; restrict outbound connections from the DMZ



Secure Development Practices

Use parameterized queries, validate input, avoid dynamic file includes, and conduct code audits



Key Takeaways

- 1 Web shells are small, easy to deploy, and extremely dangerous
- 2 Prevention starts with secure coding and strict upload validation
- 3 Detection requires continuous monitoring, not just perimeter defenses
- 4 Incident response plans should include web shell hunting procedures
- 5 Defense in depth is the only reliable strategy



Thank You

Questions & Discussion